

# COMPILER DESIGN

## PROGRAMMING EXERCISE 4

MAANYA BHARDWAJ

PES2UG23CS311

E SECTION

symbol\_table.h

```
#ifndef SYMBOL_TABLE_H
#define SYMBOL_TABLE_H

#define MAX 200

typedef struct Symbol {
    char name[50];
    char kind[30];
    char type[50];
    char storage[20];
    int size;
    char scope[20];
    int line;
    int column;
    char file[50];
    char value[50];
} Symbol;

void insert(char*, char*, char*, char*, int, char*, int, int, char*);
void display();

int lookup(char*);
void update_value(char*, int);
int get_value(char*);
char* get_type(char*);
int get_size(char*);
```

```
#endif
```

## symbol\_table.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "symbol_table.h"

Symbol table[MAX];
int count = 0;

int lookup(char* name) {
    for(int i=0;i<count;i++) {
        if(strcmp(table[i].name,name)==0)
            return i;
    }
    return -1;
}

void insert(char* name, char* kind, char* type, char* storage,
            int size, char* scope, int line, int column, char* file) {

    if(lookup(name) != -1) return;

    strcpy(table[count].name, name);
    strcpy(table[count].kind, kind);
    strcpy(table[count].type, type);
    strcpy(table[count].storage, storage);
    table[count].size = size;
    strcpy(table[count].scope, scope);
    table[count].line = line;
    table[count].column = column;
    strcpy(table[count].file, file);
    strcpy(table[count].value, "NULL");

    count++;
}

void update_value(char* name, int val) {
```

```

    int i = lookup(name);
    if(i != -1) {
        sprintf(table[i].value, "%d", val);
    }
}

int get_value(char* name) {
    int i = lookup(name);
    if(i != -1) {
        return atoi(table[i].value);
    }
    return 0;
}

char* get_type(char* name) {
    int i = lookup(name);
    if(i != -1) return table[i].type;
    return "";
}

int get_size(char* type) {
    if(strstr(type, "*")) return 8;
    if(strcmp(type, "int")==0) return 4;
    if(strcmp(type, "float")==0) return 4;
    if(strcmp(type, "char")==0) return 1;
    return 0;
}

void display() {
    printf("\nSYMBOL TABLE:\n");

    printf("-----\n");
    printf("Name\tKind\t\tType\tStorage\tSize\tScope\tLine\tCol\tValue\n");

    printf("-----\n");

    for(int i=0;i<count;i++) {
        printf("%s\t%s\t%s\t%s\t%d\t%s\t%d\t%d\t%s\n",
            table[i].name,
            table[i].kind,

```

```

        table[i].type,
        table[i].storage,
        table[i].size,
        table[i].scope,
        table[i].line,
        table[i].column,
        table[i].value
    );
}
}

```

## lexer.l

```

%{
#include "parser.tab.h"
#include <stdlib.h>
#include <string.h>

int line = 1;
%}

%%

"int"    { return INT; }
"float"  { return FLOAT; }
"char"   { return CHAR; }

"static" { return STATIC; }
"extern"  { return EXTERN; }
"register" { return REGISTER; }

"typedef" { return TYPEDEF; }
"struct"  { return STRUCT; }
"union"   { return UNION; }
"enum"    { return ENUM; }

[0-9]+   { yylval.num = atoi(yytext); return NUM; }

```

```
[a-zA-Z_][a-zA-Z0-9_]* { yyval.str = strdup(yytext); return ID; }
```

```
"+" { return PLUS; }
```

```
"-" { return MINUS; }
```

```
"*" { return MUL; }
```

```
"/" { return DIV; }
```

```
"=" { return ASSIGN; }
```

```
"(" { return LPAREN; }
```

```
")" { return RPAREN; }
```

```
"{" { return LBRACE; }
```

```
"}" { return RBRACE; }
```

```
"," { return SEMICOLON; }
```

```
"," { return COMMA; }
```

```
[ \t] ;
```

```
\n { line++; }
```

```
. { return yytext[0]; }
```

```
%%
```

```
int yywrap() { return 1; }
```

## parser.y

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include "symbol_table.h"  
  
extern int line;  
  
char current_type[50];
```

```

void yyerror(const char *s);
int yylex();
%}

%union {
    int num;
    char* str;
}

%token <str> ID
%token <num> NUM
%token INT FLOAT CHAR STATIC EXTERN REGISTER TYPEDEF STRUCT UNION ENUM
%token PLUS MINUS MUL DIV ASSIGN
%token LPAREN RPAREN LBRACE RBRACE SEMICOLON COMMA

%type <num> expr
%left PLUS MINUS
%left MUL DIV
%%

program:
    program stmt
    |
    ;

stmt:
    declaration
    | assignment
    | function
    | typedef_stmt
    | struct_stmt
    | union_stmt
    | enum_stmt
    ;

// ----- DECL -----

declaration:
    type ID SEMICOLON {
        insert($2,"variable",current_type,"auto",
            get_size(current_type),"global",line,0,"input.c");
    }

```

```

;

type:
    INT { strcpy(current_type,"int"); }
    | FLOAT { strcpy(current_type,"float"); }
    | CHAR { strcpy(current_type,"char"); }
    ;

// ----- ASSIGNMENT (WITH COMMA SUPPORT) -----

assignment:
    assign_list SEMICOLON
    ;

assign_list:
    single_assign
    | assign_list COMMA single_assign
    ;

single_assign:
    ID ASSIGN expr {
        if(lookup($1)==-1) {
            printf("Error: Undeclared variable %s\n",$1);
        } else {
            update_value($1,$3);
        }
    }
    ;

// ----- EXPRESSIONS -----

expr:
    expr PLUS expr { $$ = $1 + $3; }
    | expr MINUS expr { $$ = $1 - $3; }
    | expr MUL expr { $$ = $1 * $3; }
    | expr DIV expr { $$ = $1 / $3; }
    | LPAREN expr RPAREN { $$ = $2; }
    | NUM { $$ = $1; }
    | ID {
        if(lookup($1)==-1) {
            printf("Error: Undeclared variable %s\n",$1);
            $$ = 0;

```

```

        } else {
            if(strcmp(get_type($1),"int")!=0) {
                printf("Type Error: %s not int\n",$1);
            }
            $$ = get_value($1);
        }
    }
;

// ----- FUNCTION -----

function:
    type ID LPAREN RPAREN LBRACE RBRACE {
        insert($2,"function",current_type,"auto",0,"global",line,0,"input.c");
    }
;

// ----- TYPEDEF -----

typedef_stmt:
    TYPEDEF type ID SEMICOLON {
        insert($3,"typedef",current_type,"auto",
            get_size(current_type),"global",line,0,"input.c");
    }
;

// ----- STRUCT -----

struct_stmt:
    STRUCT ID LBRACE RBRACE SEMICOLON {
        insert($2,"struct-tag","struct","auto",0,"global",line,0,"input.c");
    }
;

// ----- UNION -----

union_stmt:
    UNION ID LBRACE RBRACE SEMICOLON {
        insert($2,"union-tag","union","auto",0,"global",line,0,"input.c");
    }
;

```



```
// ----- ENUM -----

enum_stmt:
    ENUM ID LBRACE enum_list RBRACE SEMICOLON {
        insert($2,"enum-tag","enum","auto",4,"global",line,0,"input.c");
    }
;

enum_list:
    ID {
        insert($1,"enum-const","int","auto",4,"global",line,0,"input.c");
    }
| enum_list COMMA ID {
    insert($3,"enum-const","int","auto",4,"global",line,0,"input.c");
}
;

%%

void yyerror(const char *s){
    printf("Error: %s at line %d\n",s,line);
}

int main(){
    yyparse();
    display();
    return 0;
}
```

input .c

```
int a;
int b;
int c;
int d;

a=5,b=8,c=9,d=a+b*c;
```

output :

```
maanyabhardwaj@Maanyabhardwaj PE3 % flex lexer.l
bison -d parser.y
gcc lex.yy.c parser.tab.c symbol_table.c -o symtab
./symtab < input.c
```

SYMBOL TABLE:

| Name | Kind     | Type | Storage | Size | Scope  | Line | Col | Value |
|------|----------|------|---------|------|--------|------|-----|-------|
| a    | variable | int  | auto    | 4    | global | 1    | 0   | 5     |
| b    | variable | int  | auto    | 4    | global | 2    | 0   | 8     |
| c    | variable | int  | auto    | 4    | global | 3    | 0   | 9     |
| d    | variable | int  | auto    | 4    | global | 4    | 0   | 77    |

```
maanyabhardwaj@Maanyabhardwaj PE3 %
```

input .c

int a;

int b;

int c;

int d;

int e;

a=10,b=5,c=2,d=a-b\*c,e=a/b+c;

Output:

```
d      variable      int      auto      4      global  4      0      77
```

```
maanyabhardwaj@Maanyabhardwaj PE3 % flex lexer.l
bison -d parser.y
gcc lex.yy.c parser.tab.c symbol_table.c -o symtab
./symtab < input.c
```

SYMBOL TABLE:

| Name | Kind     | Type | Storage | Size | Scope  | Line | Col | Value |
|------|----------|------|---------|------|--------|------|-----|-------|
| a    | variable | int  | auto    | 4    | global | 1    | 0   | 10    |
| b    | variable | int  | auto    | 4    | global | 2    | 0   | 5     |
| c    | variable | int  | auto    | 4    | global | 3    | 0   | 2     |
| d    | variable | int  | auto    | 4    | global | 4    | 0   | 0     |
| e    | variable | int  | auto    | 4    | global | 5    | 0   | 4     |

```
maanyabhardwaj@Maanyabhardwaj PE3 %
```